Raspberry Pi Private NAS

Technical Documentation

Raspberry Pi 4 · OpenMediaVault · LUKS · Samba · rsync

October 2026

Contents

Raspberry Pi Private NAS Server 5
- At a glance 5
- Documentation 6
- Quick start 6
- Repository layout 7
- Use cases 7
- Roadmap 7
- Disclaimer 7

Requirements 8
- Hardware 8
- Software 8
- Network 9
- Skills 9

System Architecture 10
- 1. Purpose and scope 10
- 2. Logical layers 10
- 3. Physical topology 10
- 4. Storage stack 11
- 5. Data flows 11
- 6. Key design decisions 12
- 7. Capacity and performance expectations 12
- 8. Failure domains 13
- 9. Future architecture 13

System Diagrams 14
- Overall system 14
- Storage stack 14
- Request path (SMB write) 14
- Boot sequence 15

Hardware 16
- Bill of materials 16
- Power budget 16
- Thermal 17
- Assembly 17
- Identifying drives 17

Installation Guide 18
- Step 1 — Flash Raspberry Pi OS Lite 18
- Step 2 — Find the Pi and connect 18
- Step 3 — Update the system 18
- Step 4 — Install OpenMediaVault 18
- Step 5 — First login to the web UI 19
- Step 6 — Install plugins 19

Step 7 — Storage, shares, users, backups 19
Step 8 — Acceptance checklist 19
Storage & Encryption 21
1. Identify the data disk 21
2. Choose a method 21
3A. Encrypt via OMV 21
3B. Encrypt via CLI 21
4. Unlocking after reboot 22
5. Header backup (do this now) 22
6. Backup disk 22
7. Convenience symlinks (optional) 22
8. Performance tuning 23
9. Verification 23
Users & Permissions 24
Roles 24
Groups (recommended pattern) 24
Create a user 24
Grant share access 24
Per-user private folders 25
Password policy 25
Audit 25
SMB Network Sharing 26
1. Create the shared folder 26
2. Enable SMB 26
3. Add the share 26
4. Grant access 27
5. Connect from clients 27
6. Reference: effective share stanza 27
7. Verify 28
Backup & Restore 29
1. Design 29
2. Install the script 29
3. Schedule 29
4. Verification 30
5. Versioned snapshots (recommended) 30
6. Restore procedures 30
7. OMV configuration backup 31
Security Hardening 32
1. Threat model 32
2. OMV admin UI 32
3. SSH 32
4. Firewall 33
5. SMB 33
6. Updates 33
7. Secrets handling 33

8. Periodic review (quarterly) 34
Remote Access 35
Options 35
WireGuard on the Pi 35
Revoking access 36
Operations & Maintenance 37
Routine schedule 37
After every reboot 37
Monitoring 37
Updates 38
Config backup 38
Reducing SD-card wear 38
Clean shutdown 38
Capacity expansion 38
Troubleshooting 39
USB enclosure UAS quirk 39
Useful logs 40
Reset OMV admin password (lost) 40
Scripts Reference 41
`scripts/encryption-setup.sh` (recommended) 41
`scripts/disk-setup.sh` 41
`scripts/backup-cron.sh` 41
Changes from the original scripts 42
Script: scripts/backup-cron.sh 43
Script: scripts/disk-setup.sh 44
Script: scripts/encryption-setup.sh 45

Raspberry Pi Private NAS Server

A secure, low-power Network Attached Storage (NAS) built on a Raspberry Pi 4 and OpenMediaVault (OMV). It works as a private home cloud: encrypted storage, SMB file sharing to Windows/macOS/Linux, per-user permissions and automated nightly backups.

At a glance

Item	Choice
Compute	Raspberry Pi 4 Model B (4 GB+ recommended)
Primary storage	USB 3.0 SSD, 500 GB+, LUKS-encrypted, ext4
Backup storage	Second USB drive (separate physical disk)
OS	Raspberry Pi OS Lite (64-bit)
NAS platform	OpenMediaVault + `openmediavault-luksencryption` plugin
File sharing	Samba (SMB 2/3)
Backups	rsync on a cron schedule, with logging and verification
Admin access	OMV web UI (LAN only), SSH with keys

Documentation

Doc	What it covers
Architecture	Layers, components, data flow, design decisions
Hardware	Bill of materials, power, cooling, assembly
Installation Guide	OS → OMV → first login, end to end
Storage & Encryption	Disk prep, LUKS, ext4, mounting in OMV, unlocking after reboot
SMB Sharing	Shared folders, Samba settings, connecting from each OS
Users & Permissions	Roles, groups, ACLs, guest access
Backup & Restore	Backup design, schedule, verification, restore procedures
Security Hardening	SSH, firewall, OMV admin, updates, threat model
Remote Access	Accessing the NAS away from home safely (VPN)
Operations & Maintenance	Routine tasks, monitoring, updates, disk health
Troubleshooting	Common faults and fixes
Scripts Reference	What each script does, arguments, safety notes
System Diagram	ASCII architecture and data-flow diagrams

Quick start

1. Flash Raspberry Pi OS Lite (64-bit) with SSH enabled.
2. `sudo apt update && sudo apt full-upgrade -y`
3. Install OMV with the community install script (see **Installation Guide**).
4. Log in to `http://<pi-ip>/`, change the admin password immediately.
5. Encrypt and mount the data SSD (**Storage & Encryption**).
6. Create users and the `nas_storage` share (**Users**, **SMB**).
7. Attach the backup drive and enable the nightly job (**Backup**).
8. Harden SSH and the firewall (**Security**).

Repository layout

```
.
├── README.md
├── requirements.md
├── config/          # Example SMB and permission settings
├── diagrams/        # Architecture diagrams
├── docs/            # Technical documentation (this set)
├── hardware/        # Component notes
└── scripts/         # Disk, encryption and backup helpers
```

Use cases

Personal cloud storage · media library · development backup target · secure document archive.

Roadmap

- Nextcloud (or OMV’s compose plugin) for web/mobile file access
- RAID 1 / SnapRAID on a powered USB hub or SATA HAT
- WireGuard VPN for remote access (documented in **Remote Access**)
- SMART + email alerts, uptime monitoring

Disclaimer

The scripts format and encrypt disks. Running them against the wrong device destroys data. Read **Scripts Reference** first.

Requirements

Hardware

Component	Minimum	Recommended
Raspberry Pi	Pi 4 Model B, 2 GB	Pi 4 Model B, 4 GB or 8 GB
Boot media	16 GB microSD (A1)	32 GB A2 microSD, or boot from a small USB SSD
Data drive	500 GB SSD	1 TB+ SATA SSD
Enclosure	USB 3.0 SATA	USB 3.0 with UASP, known-good chipset (e.g. ASMedia ASM1153E/ ASM235CM)
Backup drive	Any USB drive ≥ data used	Same or larger capacity than data drive
Power	Official 5.1 V / 3 A USB-C PSU	Official PSU; powered USB hub if running two drives that draw > 1.2 A combined
Network	Gigabit Ethernet cable	Wired to router/switch (Wi-Fi not recommended)
Cooling	Passive heatsinks	Case with fan or aluminium heatsink case

Software

Package	Purpose
Raspberry Pi OS Lite (64-bit)	Base OS. The Lite (no desktop) image is required by the OMV install script.
OpenMediaVault	Web-managed NAS platform (installs Samba, NFS, rsync, cron management)
openmediavault-luksencryption	OMV plugin to create/unlock LUKS volumes from the web UI
cryptsetup	LUKS tooling (pulled in by the plugin)
Samba	SMB file sharing
rsync	Backups
smartmontools	Disk health monitoring
openssh-server	Remote administration

Check the OMV install script’s README for the Raspberry Pi OS release it currently supports before flashing.

Network

- Static IP or DHCP reservation for the Pi
- Router access (for the reservation, and for VPN port forwarding if used)
- Client devices on the same LAN for SMB

Skills

Comfort with a Linux shell, SSH and editing files with `nano`/`vim`.

System Architecture

1. Purpose and scope

The system provides private, encrypted network storage for a home or small office LAN. It is a single-node design: one Raspberry Pi, one primary data disk, one backup disk. It is not a high-availability system.

2. Logical layers

Layer	Components	Responsibility
Access	Samba (SMB 2/3), OMV web UI (HTTP/HTTPS), SSH, optional WireGuard	How clients and admins reach the NAS
Management	OpenMediaVault (Salt-based config, `omv-engined`), cron, OMV plugins	Single source of truth for shares, users, mounts, jobs
Security	LUKS2 (dm-crypt), Linux users/groups, POSIX ACLs, nftables/ufw, SSH keys	Confidentiality at rest, authentication, authorization, network filtering
Storage	USB 3.0 SSD → LUKS container → ext4; second USB disk for backup	Persistence
Platform	Raspberry Pi 4, Raspberry Pi OS Lite (Debian-based)	Hardware and kernel

3. Physical topology

```
[Clients] ──LAN── [Router/Switch] ──GbE── [Raspberry Pi 4]
                                            ├─ USB3 ── SSD (data, LUKS → ext4)  /srv/dev-disk-by-uuid-<data>
                                            ├─ USB3 ── HDD/SSD (backup, ext4)    /srv/dev-disk-by-uuid-<backup>
                                            └─ microSD (OS)
```

Use the **blue USB 3.0 ports** for both drives. The Pi 4's two USB 3 ports share one controller (~350–400 MB/s total), so Gigabit Ethernet (~110 MB/s) is normally the bottleneck, not USB.

4. Storage stack

```
/dev/sda (SSD)
 └─ /dev/sda1  (GPT partition)
     └─ LUKS2 container
         └─ /dev/mapper/sda1-crypt  (opened mapping)
             └─ ext4  ── mounted by OMV at /srv/dev-disk-by-uuid-<UUID>
                 └─ shared folder "nas_storage" ── exposed by Samba as \\<host>\nas_storage
```

Mount paths. OMV mounts disks under `/srv/dev-disk-by-uuid-<UUID>`, not `/mnt`. Scripts and backup jobs must reference these paths (or a symlink), otherwise they write to the SD card. The original repo used `/mnt/nas-storage` and `/data/storage` interchangeably; this doc set standardises on OMV paths, with optional convenience symlinks `/mnt/nas-storage` and `/mnt/nas-backup`.

5. Data flows

File read/write (SMB): client → TCP 445 → `smbd` authenticates against Samba user DB (synced by OMV) → checks share ACL + filesystem permissions → reads/writes ext4 → dm-crypt encrypts blocks → SSD.

Administration: admin browser → TCP 80/443 → nginx → OMV PHP-FPM/RPC → `omv-engined` → Salt applies config to Samba, fstab, cron, etc.

Backup: cron (02:00) → `backup-cron.sh` → checks both mounts → `rsync -aHAX --delete` data → backup disk → writes log + status file.

6. Key design decisions

Decision	Rationale	Trade-off
OMV instead of hand-configured Samba	Web UI, consistent config, plugins	OMV owns config files; manual edits to `smb.conf`/`fstab` get overwritten
LUKS on data disk	Protects data if disk or Pi is stolen	Disk must be unlocked after every reboot (manual or keyfile)
ext4	Mature, low overhead on the Pi	No checksums/snapshots (btrfs would add them at CPU cost)
Separate backup disk	Survives data-disk failure	Same location: not protected from fire/theft without an off-site copy
rsync mirror with `--delete`	Simple, fast, restorable with any tool	A mirror replicates accidental deletions at the next run; see versioning option in Backup doc
Wired Ethernet	Throughput and stability	Pi must be near router/switch
No public exposure of SMB/web UI	SMB and admin UIs are frequent attack targets	Remote access requires a VPN

7. Capacity and performance expectations

- SMB throughput over GbE: roughly 70–110 MB/s sequential, lower for many small files.
- LUKS on the Pi 4 has no AES hardware acceleration; expect throughput reduction. If it bottlenecks, benchmark `cryptsetup benchmark` and consider `xchacha20,aes-adiantum-plain64` (Adiantum), which is designed for CPUs without AES instructions.
- Concurrent users: a handful of simultaneous clients is comfortable.

8. Failure domains

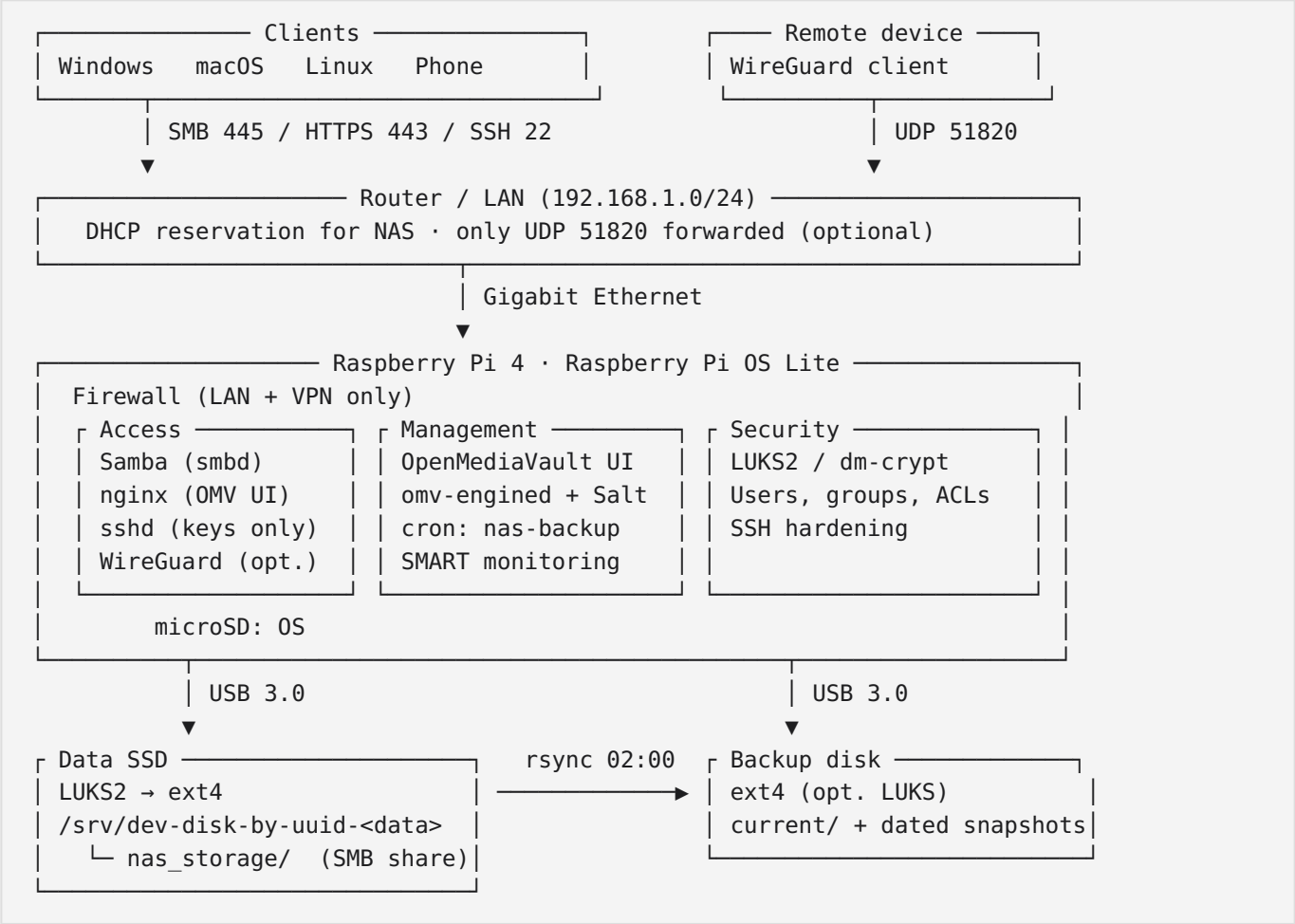
Failure	Impact	Mitigation
SD card corruption	NAS down, data intact	Backup OMV config; keep a spare imaged card; consider USB boot
Data SSD failure	Data unavailable	Restore from backup disk
Backup disk failure	Loss of backup only	SMART monitoring, replace
Power loss	Possible fs inconsistency	UPS or at least a good PSU; ext4 journaling
Theft of drives	None if locked	LUKS; never store the keyfile on the same disk
Ransomware on a client	Encrypted files synced to backup at next run	Versioned/snapshot backups, restricted write access

9. Future architecture

- RAID 1 (via powered hub) or SnapRAID + mergerfs for redundancy
- Nextcloud via OMV compose plugin for web/mobile access
- WireGuard for remote access
- Off-site encrypted backup (restic/borg to a cloud bucket or second location)

System Diagrams

Overall system



Storage stack

```
/dev/sda → /dev/sda1 → [LUKS2] → /dev/mapper/sda1-crypt → ext4 → /srv/dev-disk-by-uuid-… →
nas_storage → \\nas\nas_storage
```

Request path (SMB write)

```
Client —SMB→ smbd —auth→ Samba user DB
                 └─authz→ share ACL + POSIX perms → ext4 → dm-crypt (encrypt) → SSD
```

Boot sequence

```
Power on → OS boots → OMV services start → LUKS volume LOCKED
        → admin unlocks (UI/SSH or keyfile) → ext4 mounted → SMB share available
```

Hardware

Bill of materials

#	Component	Spec	Notes
1	Raspberry Pi 4 Model B	4 GB RAM (2 GB works)	8 GB only useful if you add containers
2	Power supply	Official 5.1 V 3 A USB-C	Under-voltage is the #1 cause of USB disk drop-outs
3	microSD	32 GB, A1/A2, reputable brand	Holds OS only
4	Data SSD	2.5” SATA SSD, 500 GB+	
5	USB 3.0 SATA enclosure	UASP support	Avoid JMicron JMS578/567 if you see resets; see Troubleshooting
6	Backup drive	≥ data capacity	2.5” HDD fine; 3.5” needs own power
7	Powered USB 3.0 hub	Optional	Required if both drives are bus-powered HDDs
8	Ethernet cable	Cat5e or better	
9	Case + cooling	Heatsink case or fan	Pi 4 throttles at ~80–85 °C
10	UPS	Optional	Clean shutdown on power loss

Power budget

The Pi 4’s USB ports supply a combined ~1.2 A. A SATA SSD typically draws 0.5–1 A under load; a 2.5” HDD can spike above 1 A at spin-up. Two bus-powered spinning disks on the Pi itself are not reliable — use a powered hub or self-powered enclosure.

Check for under-voltage at any time:

```
        vcgencmd get_throttled
# 0x0 = healthy. Bit 0 (0x1) = under-voltage now; bit 16 (0x10000) = has occurred
```

Thermal

```
vcgencmd measure_temp
```

Target < 70 °C sustained. Keep the case ventilated; SSDs in enclosures also run warm.

Assembly

1. Fit heatsinks/case.
2. Flash the microSD (see Installation Guide) and insert.
3. Install the SSD into the USB enclosure.
4. Connect Ethernet.
5. Connect the data SSD to a **blue USB 3.0 port**. Connect the backup drive to the other blue port (or the powered hub).
6. Connect power last.

Identifying drives

```
lsblk -o NAME,SIZE,MODEL,SERIAL,TRAN,FSTYPE,MOUNTPOINT
ls -l /dev/disk/by-id/
```

Device names (`/dev/sda`, `/dev/sdb`) can swap between boots. Always identify disks by serial/`by-id` before formatting and refer to filesystems by UUID.

Installation Guide

Estimated time: 60–90 minutes. Everything after Step 3 can be done from another computer.

Step 1 — Flash Raspberry Pi OS Lite

1. Install **Raspberry Pi Imager** on your computer.
2. Choose *Raspberry Pi 4* → *Raspberry Pi OS (other)* → **Raspberry Pi OS Lite (64-bit)**. The desktop image is not supported by the OMV installer.
3. Open *Edit settings* (OS customisation):
    - Hostname: `nas` (or keep `raspberrypi`)
    - Create a user (do **not** use `pi`/`raspberry`); strong password
    - Locale/timezone
    - **Services → Enable SSH**, ideally "Allow public-key authentication only" and paste your public key
    - Leave Wi-Fi unset — the NAS should be wired
4. Write the card, insert it, connect Ethernet and power.

Step 2 — Find the Pi and connect

Check your router's DHCP list, or:

```
ping nas.local
ssh <user>@nas.local
```

Create a **DHCP reservation** in your router for the Pi's MAC address so its IP never changes.

Step 3 — Update the system

```
sudo apt update
sudo apt full-upgrade -y
sudo reboot
```

Step 4 — Install OpenMediaVault

Use the community-maintained installer (OMV-Plugin-Developers). Review it first if you prefer:

```
        wget -O install-omv.sh https://github.com/OpenMediaVault-Plugin-Developers/installScript/
raw/master/install
        less install-omv.sh
        sudo bash install-omv.sh -n
```

Notes: - Run it over **wired** Ethernet. The script reconfigures networking and the connection may drop; reconnect after it finishes. - `-n` skips network reconfiguration if you want to keep the existing setup (see the script's README for current flags). - The run takes 15–30 minutes. Reboot when done: `sudo reboot`.

Step 5 — First login to the web UI

Browse to `http://<pi-ip>/` (or `http://nas.local/`).

Default credentials:

```
username: admin
password: openmediavault
```

Immediately: top-right user menu → *Change Password*. Then:

- *System* → *General Settings*: set session timeout (e.g. 5 min).
- *System* → *Date & Time*: set timezone, enable NTP.
- *System* → *Notification*: configure SMTP so OMV can email you alerts.

Step 6 — Install plugins

System → *Plugins*: - `openmediavault-luksencryption` (required for encryption) - Optional: `openmediavault-flashmemory` — reduces SD-card writes (follow its post-install instructions)

Step 7 — Storage, shares, users, backups

Continue in order:

1. **Storage & Encryption**
2. **Users & Permissions**
3. **SMB Sharing**
4. **Backup & Restore**
5. **Security Hardening**

Step 8 — Acceptance checklist

☐

- [ ] Admin password changed, default `admin/openmediavault` no longer works
- [ ] Data disk is LUKS-encrypted, unlocks, and mounts under `/srv/dev-disk-by-uuid-…`
- [ ] Share visible from Windows and macOS, write test succeeds as a normal user
- [ ] Guest (if disabled) cannot connect
- [ ] Backup job ran manually and its log shows success
- [ ] `ssh root@nas` refused; password SSH refused
- [ ] `vcgencmd get_throttled` returns `0x0`
- [ ] OMV configuration backed up off the Pi

Storage & Encryption

> **Destructive.** These steps erase the target disk. Confirm the device by serial number before every format.

1. Identify the data disk

```
lsblk -o NAME,SIZE,MODEL,SERIAL,TRAN,FSTYPE,MOUNTPOINT
```

Assume `/dev/sda` below; substitute your own.

2. Choose a method

Method	Use when
A. OMV web UI (recommended)	Normal setup; OMV tracks everything
B. Command line (`scripts/encryption-setup.sh`)	Scripted/rebuild; then import into OMV

Do **not** run both `disk-setup.sh` and `encryption-setup.sh`. The first creates a plain ext4 filesystem; the second creates an encrypted one. Pick encrypted.

3A. Encrypt via OMV

1. *Storage* → *Disks*: select the SSD → **Wipe** (Quick).
2. *Storage* → *Encryption* → **+ Create**: device = the SSD, set a strong passphrase (store it in a password manager — there is no recovery).
3. Select the new encrypted device → **Unlock** → enter passphrase.
4. *Storage* → *File Systems* → **+ Create and mount**: device = `…-crypt`, type **EXT4**. Wait for formatting.
5. Click **Apply** (yellow banner) to save.

3B. Encrypt via CLI

```
sudo ./scripts/encryption-setup.sh /dev/sda
```

The script partitions the disk (GPT, one partition), runs `cryptsetup luksFormat --type luks2`, opens it as `sda1-crypt`, and creates ext4 labelled `nasdata`. Then in OMV: *Storage* → *Encryption* (it appears as locked/unlocked) → *File Systems* → **Mount** existing → Apply.

4. Unlocking after reboot

LUKS volumes stay locked after a reboot. Shares are unavailable until unlocked.

Option 1 — Manual (most secure): after each boot, *Storage* → *Encryption* → *Unlock*, then *File Systems* → *mount* if not auto-mounted. Or via SSH:

```
sudo cryptsetup open /dev/disk/by-uuid/<LUKS-UUID> sda1-crypt
sudo mount -a   # or re-apply in OMV
```

Option 2 — Keyfile on separate removable media (convenience): add a second key from a USB stick, reference it in `/etc/crypttab`. Only secure if the stick is removed/kept separately; a keyfile on the SD card means anyone stealing the whole Pi gets the data.

```
sudo dd if=/dev/urandom of=/media/keyusb/nas.key bs=512 count=8
sudo chmod 0400 /media/keyusb/nas.key
sudo cryptsetup luksAddKey /dev/sda1 /media/keyusb/nas.key
```

Keep the passphrase slot as a fallback.

5. Header backup (do this now)

If the LUKS header is damaged, the data is gone permanently. Back it up off the Pi:

```
sudo cryptsetup luksHeaderBackup /dev/sda1 --header-backup-file ~/nas-luks-header.img
# copy it to an encrypted location off the NAS, then delete the local copy
```

6. Backup disk

The backup disk can be encrypted the same way (recommended if it leaves the house) or plain ext4. Mount it in OMV like the data disk. It must be a **different physical disk** from the data disk.

7. Convenience symlinks (optional)

Scripts default to `/mnt/nas-storage` and `/mnt/nas-backup`. Point these at OMV's mounts:

```
        sudo ln -sfn /srv/dev-disk-by-uuid-<DATA-UUID>   /mnt/nas-storage
        sudo ln -sfn /srv/dev-disk-by-uuid-<BACKUP-UUID> /mnt/nas-backup
```

Find UUIDs with `ls /srv/` or `blkid`.

8. Performance tuning

```
        cryptsetup benchmark
```

If `aes-xts` is slow (no AES instructions on the Pi 4's CPU), Adiantum is a valid alternative at format time:

```
        cryptsetup luksFormat --type luks2 --cipher xchacha20,aes-adiantum-plain64 --key-size 256 /
dev/sda1
```

9. Verification

```
        sudo cryptsetup status sda1-crypt     # shows cipher, device, "active"
        lsblk -f                               # sda1 crypto_LUKS → sda1-crypt ext4
        findmnt | grep dev-disk-by-uuid
```

Users & Permissions

All accounts are managed in OMV (*Users → Users / Groups*). OMV creates the Linux account and the matching Samba password together.

Roles

Role	How it's implemented	Access
Admin	OMV `admin` web account + one Linux user in `sudo` and `_ssh`	Full system: web UI, SSH, all shares
User	OMV user in group `users` (+ functional groups)	Only shared folders explicitly granted
Guest	Disabled by default. If needed: a separate read-only share with *Public = Guests allowed*	Read-only, one share only

The OMV web `admin` account is separate from Linux users. Normal users cannot log in to the admin UI.

Groups (recommended pattern)

Grant permissions to groups, not individuals:

Group	Purpose
`nas-rw`	Read/write on `nas_storage`
`nas-ro`	Read-only on `nas_storage`
`_ssh`	Allowed to SSH (admins only)

Create a user

Users → Users → + Create: name, email, strong password, groups (`users`, `nas-rw`). Shell: `/usr/sbin/nologin` for file-only users.

Grant share access

Storage → Shared Folders → select → Permissions: set group `nas-rw` = Read/Write, `nas-ro` = Read-only. Apply.

Permissions are evaluated as: SMB share settings → OMV share privileges → filesystem permissions/ACLs. The most restrictive wins.

Per-user private folders

For one private folder per person, create shared folders `home_alice`, `home_bob`, each granting only that user, or enable *Users* → *Settings* → *User home directory* on the data disk.

Password policy

- Minimum 12 characters; unique per user
- Change on suspected compromise; remove leavers immediately (*Users* → *delete*, then check share ACLs)

Audit

```
getent group nas-rw            # who has write
sudo pdbedit -L                # Samba users
sudo smbstatus -b              # who is connected
getfacl /srv/dev-disk-by-uuid-<UUID>/nas_storage
```

SMB Network Sharing

1. Create the shared folder

Storage → Shared Folders → + Create

Field	Value
Name	`nas_storage`
File system	the encrypted data filesystem
Relative path	`nas_storage/`
Permissions	Administrator: r/w, Users: r/w, Others: no access

Apply.

2. Enable SMB

Services → SMB/CIFS → Settings

Setting	Value	Why
Enabled	✔	
Workgroup	`WORKGROUP`	Windows default
Minimum protocol	**SMB2** (or SMB3 if all clients support it)	SMB1 is insecure
Browsable	✔	
Extra options	see below	

Recommended extra options:

```
server signing = mandatory
smb encrypt = desired
restrict anonymous = 2
```

`smb encrypt = required` forces in-transit encryption but costs throughput on the Pi; use `desired` on a trusted LAN.

3. Add the share

Services → SMB/CIFS → Shares → + Create

Field	Value
Shared folder	`nas_storage`
Public	**No**
Read only	No
Browseable	Yes
Inherit ACLs / permissions	Yes
Recycle bin	Yes (retention e.g. 30 days) — protects against accidental deletes
Hide dot files	Yes

Apply.

4. Grant access

Storage → *Shared Folders* → *nas_storage* → *Permissions*: give each user or group Read/Write or Read-only. See **Users & Permissions**.

5. Connect from clients

Windows: File Explorer → `\\nas\nas_storage` (or `\\<pi-ip>\nas_storage`). To map: *This PC* → *Map network drive*, tick *Connect using different credentials*.

```
net use Z: \\nas\nas_storage /user:alice /persistent:yes
```

macOS: Finder → *Go* → *Connect to Server* (⌘K) → `smb://nas.local/nas_storage`.

Linux:

```
sudo apt install cifs-utils
sudo mount -t cifs //nas.local/nas_storage /mnt/nas \
  -o credentials=/home/$USER/.smbcred,uid=$(id -u),gid=$(id -g),vers=3.0
# ~/.smbcred:  username=alice  password=...   (chmod 600)
```

iOS/Android: Files app (iOS) “Connect to Server” or any SMB client app, on the same LAN or over VPN.

6. Reference: effective share stanza

OMV generates `/etc/samba/smb.conf`; do not edit it by hand. The resulting share looks roughly like:

```
[nas_storage]
path = /srv/dev-disk-by-uuid-<UUID>/nas_storage/
guest ok = no
read only = no
browseable = yes
inherit acls = yes
inherit permissions = yes
vfs objects = recycle
recycle:repository = .recycle/%U
```

7. Verify

```
smbclient -L //localhost -U alice      # lists nas_storage
testparm -s                            # config syntax check
sudo smbstatus                         # current sessions and locks
```

Backup & Restore

1. Design

Property	Value
Tool	rsync
Source	data filesystem (`/mnt/nas-storage` → OMV mount)
Target	separate physical backup disk (`/mnt/nas-backup`)
Mode	Mirror (`--delete`) — optionally with dated snapshots, see §5
Schedule	Daily 02:00
Logging	`/var/log/nas-backup.log`
Verification	Mount checks before run, exit-code check, monthly checksum dry run

The original `rsync -av --delete $SOURCE $DEST` (no trailing slash) copies the directory *into* `DEST/nas-storage/`, has no safety checks, and — if the backup disk isn't mounted — fills the SD card. The updated `scripts/backup-cron.sh` fixes this.

3-2-1 reminder

This design gives 2 copies on 2 disks in 1 location. For real protection add an off-site copy (encrypted restic/borg to cloud, or a rotated disk kept elsewhere).

2. Install the script

```
sudo install -m 0750 scripts/backup-cron.sh /usr/local/sbin/nas-backup
sudo nas-backup              # first run manually; can take hours
tail -n 50 /var/log/nas-backup.log
```

3. Schedule

Via OMV (recommended): *System* → *Scheduled Tasks* → + *Create*: time `0 2 * * *`, user `root`, command `/usr/local/sbin/nas-backup`, enable *Send command output via email*.

Via cron:

```
echo '0 2 * * * root /usr/local/sbin/nas-backup' | sudo tee /etc/cron.d/nas-backup
```

OMV also has a built-in *Services → Rsync → Tasks* UI that can do the same local job without a script; either is fine, but use only one.

4. Verification

- **Each run:** script exits non-zero and logs `FAILED` on error; OMV emails output.
- **Monthly:** checksum comparison (slow, read-only):

```
sudo rsync -aHAXn --checksum --delete --itemize-changes /mnt/nas-storage/ /mnt/nas-backup/current/ | head
```

Empty output = identical.

- **Quarterly:** do a real test restore of a random folder (§6).

5. Versioned snapshots (recommended)

A plain mirror propagates accidental deletes and ransomware-encrypted files at the next run. The script supports `SNAPSHOTS=1`, which uses `--link-dest` to keep dated, hard-linked snapshots that cost space only for changed files:

```
/mnt/nas-backup/
├── current -> 2026-10-18
├── 2026-10-16/
├── 2026-10-17/
└── 2026-10-18/
```

Set `KEEP_DAYS` (default 14) to prune older snapshots.

6. Restore procedures

Single file/folder:

```
sudo rsync -aHAX /mnt/nas-backup/current/nas_storage/Projects/report.docx \
                 /mnt/nas-storage/nas_storage/Projects/
```

Or from an older snapshot: replace `current` with the date.

Also check the SMB recycle bin first: `nas_storage/.recycle/<user>/`.

Full restore after data-disk failure:

1. Replace the SSD; encrypt and create the filesystem (**Storage & Encryption**).
2. Re-create the shared folder in OMV with the same relative path.
3. Copy back:

```
sudo rsync -aHAX --info=progress2 /mnt/nas-backup/current/ /mnt/nas-storage/
```

4. Re-apply permissions in OMV (*Shared Folders* → *ACL* → *Apply recursively*) if needed.
5. Update the symlink `/mnt/nas-storage` to the new UUID.

Full rebuild after SD-card failure: re-flash, reinstall OMV, restore OMV configuration (see Operations doc), unlock the data disk — data is untouched.

7. OMV configuration backup

Keep a copy of `/etc/openmediavault/config.xml` off the Pi after every change, or use the `openmediavault-backup` plugin to image the OS disk to the backup drive weekly.

Security Hardening

1. Threat model

Threat	Control
Physical theft of disk or Pi	LUKS encryption, no keyfile stored on the Pi
Unauthorised LAN user	Per-user SMB auth, no guest, firewall
Internet attackers	Nothing exposed except optional WireGuard (UDP)
Credential guessing on SSH	Key-only auth, no root login, fail2ban
Default credentials	OMV admin password changed at install
Ransomware on a client	Least-privilege write access, recycle bin, versioned backups
Unpatched software	Scheduled updates

2. OMV admin UI

- Change `admin` password at first login.
- *System* → *Workbench*: enable **HTTPS** with a self-signed certificate (*System* → *Certificates* → *SSL* → *Create*), tick *Force SSL*.
- Set auto-logout to 5 minutes.
- Never port-forward 80/443 to the Pi.

3. SSH

Edit via OMV (*Services* → *SSH*): disable *Permit root login*, disable *Password authentication*, enable *Public key authentication*. Add your public key under *Users* → *Users* → *(user)* → *SSH keys*, and add the user to group `_ssh`.

Equivalent `sshd_config` result:

```
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
AllowGroups _ssh
```

Test in a **second** session before closing the first.

Optional brute-force protection:

```
sudo apt install fail2ban
```

4. Firewall

Manage in *System → Network → Firewall* (OMV writes iptables/nftables rules) so it survives config changes. Policy: allow from the LAN subnet only, drop the rest.

Port	Proto	Service	Source
22	TCP	SSH	LAN (admin host ideally)
80/443	TCP	OMV web UI	LAN
445	TCP	SMB	LAN + VPN subnet
51820	UDP	WireGuard (if used)	Any
—	—	everything else	drop

Example rules (LAN = `192.168.1.0/24`): accept established/related; accept loopback; accept 22, 80, 443, 445 from `192.168.1.0/24`; final rule drop. Add rules from the top down and keep an SSH session open while testing.

Disable unused services: NFS, FTP, rsync daemon, etc. under *Services*.

5. SMB

Minimum protocol SMB2+, `server signing = mandatory`, `restrict anonymous = 2`, no public shares. See **SMB Sharing**.

6. Updates

System → Update Management: check weekly, or enable unattended security updates:

```
sudo apt install unattended-upgrades
sudo dpkg-reconfigure -plow unattended-upgrades
```

Update OMV plugins through the UI. Reboot after kernel updates (and then unlock the LUKS disk).

7. Secrets handling

- LUKS passphrase and LUKS header backup: password manager / offline, never on the NAS.
- No credentials in scripts committed to Git.
- SMB credentials files on clients: `chmod 600`.

8. Periodic review (quarterly)

- [ ] Users and group memberships still correct
- [ ] No unexpected ports: `sudo ss -tulpn`
- [ ] Failed logins: `sudo journalctl -u ssh --since "-30d" | grep -i fail`
- [ ] Updates current
- [ ] Router has no port forwards to the NAS other than WireGuard

Remote Access

Rule: never expose SMB (445) or the OMV web UI to the internet. Reach the NAS remotely through a VPN.

Options

Option	Pros	Cons
WireGuard on the Pi (recommended)	Fast, simple, one UDP port	Needs a port forward and a stable address (public IP or dynamic DNS)
WireGuard/VPN on the router	Nothing extra on the Pi	Router must support it
Tailscale (mesh, WireGuard-based)	No port forwarding, works behind CGNAT	Third-party coordination service

WireGuard on the Pi

1. Install the OMV WireGuard plugin (*System → Plugins → openmediavault-wireguard*) or `sudo apt install wireguard`.
2. Create a tunnel on `wg0`, subnet `10.8.0.0/24`, listen port `51820/UDP`.
3. Add one client (peer) per device; scan the generated QR code in the WireGuard mobile app, or import the `.conf` on a laptop.
4. Router: forward **UDP 51820** → Pi's LAN IP. If your ISP changes your IP, set up dynamic DNS.
5. Firewall: allow UDP 51820 from anywhere; allow 445 (and 443 if desired) from `10.8.0.0/24`.
6. Client `AllowedIPs`: `10.8.0.0/24, 192.168.1.0/24` (split tunnel) so only NAS traffic uses the VPN.

Example peer config:

```
[Interface]
PrivateKey = <client-private-key>
Address = 10.8.0.2/32
DNS = 192.168.1.1

[Peer]
PublicKey = <server-public-key>
Endpoint = <your-ddns-hostname>:51820
AllowedIPs = 10.8.0.0/24, 192.168.1.0/24
PersistentKeepalive = 25
```

Then connect to `\\192.168.1.x\nas_storage` as on the LAN. If you’re behind carrier-grade NAT (no public IPv4), use Tailscale instead.

Revoking access

Remove the peer from the tunnel and apply. Rotate keys if a device is lost.

Operations & Maintenance

Routine schedule

Frequency	Task
Daily (auto)	Backup at 02:00; check email notification
Weekly	Glance at OMV Dashboard (disk usage, temperature, services); apply updates
Monthly	SMART check; checksum backup verification; review logs
Quarterly	Test restore; security review; export OMV config
Yearly	Replace backup disk if old/aging; review capacity

After every reboot

1. Unlock the LUKS volume (*Storage → Encryption → Unlock*) unless using a keyfile.
2. Confirm the filesystem is mounted and SMB is running.

Monitoring

OMV: *System → Monitoring* (enable), *Diagnostics → System Information / Services / System Logs*. Configure email in *System → Notification* so failures reach you.

Disk health (SMART): enable *Storage → S.M.A.R.T.*, turn on monitoring for each disk, schedule a weekly short self-test. Many USB bridges need `-d sat`:

```
sudo smartctl -a -d sat /dev/sda
```

Watch: Reallocated sectors, Pending sectors, Media wearout / Percentage used, CRC errors (often a bad cable/enclosure).

Quick health commands:

```
df -h | grep srv                # capacity
vcgencmd measure_temp
vcgencmd get_throttled          # 0x0 is healthy
systemctl --failed
journalctl -p err -b
tail -n 30 /var/log/nas-backup.log
```

Alert threshold: keep data disk below 80% full.

Updates

```
sudo omv-upgrade      # OMV's wrapper for apt full-upgrade
```

or *System → Update Management*. Reboot for kernel updates.

Config backup

```
sudo cp /etc/openmediavault/config.xml ~/omv-config-$(date +%F).xml
```

Copy off the Pi. For full OS images, use the `openmediavault-backup` plugin targeting the backup disk.

Reducing SD-card wear

Install `openmediavault-flashmemory`, keep logs small, or move the OS to a USB SSD boot (update the Pi 4 bootloader EEPROM via `sudo rpi-eeprom-update` / `raspi-config` boot order).

Clean shutdown

```
sudo poweroff
```

Never pull power while users are writing. Wait for the green LED to stop flashing.

Capacity expansion

1. Larger SSD: add it, encrypt, create fs, rsync data over, repoint shared folder, retire old disk.
2. More disks: use a powered hub; consider mergerfs (pooling) + SnapRAID (parity) plugins.

Troubleshooting

Symptom	Likely cause	Fix
Can’t reach web UI after OMV install	Install script changed networking / new IP	Check router DHCP list; connect a monitor; `ip a`
Share missing after reboot	LUKS volume locked	Unlock in *Storage* → *Encryption*, then mount
`\\nas` not found on Windows	Name resolution; NetBIOS off	Use IP `\\192.168.1.x\nas_storage`; enable WSDD in OMV SMB settings
“Access denied” on share	User lacks share privilege, or fs perms	*Shared Folders* → *Permissions*; reset ACL recursively
Windows keeps old credentials	Cached session	`net use * /delete` then reconnect
Slow transfers (< 30 MB/s)	Drive on USB 2 port; Wi-Fi; `smb encrypt = required`; LUKS cipher	Blue port; wired; `desired`; benchmark ciphers
Disk disconnects, I/O errors in `dmesg`	Under-voltage, UAS bug in enclosure	`vcgencmd get_throttled`; official PSU/powered hub; UAS quirk (below)
Backup filled the SD card	Backup disk not mounted when job ran	Updated script checks mounts; clean up `/mnt/nas-backup` on the root fs
Backup log shows `code 23`	Some files couldn’t be read (permissions/locks)	Check listed paths; usually harmless if transient
Read-only filesystem	ext4 errors after power loss	Unmount, `sudo fsck.ext4 -f /dev/mapper/sda1-crypt`, remount
High temperature / throttling	Poor airflow	Heatsink/fan case
Forgot LUKS passphrase	—	No recovery. Restore from backup onto a re-encrypted disk

USB enclosure UAS quirk

Some enclosures (notably certain JMicron bridges) reset under load with UAS. Disable UAS for that device:

```
lsusb                        # note ID, e.g. 152d:0578
sudo sed -i '1 s/$/ usb-storage.quirks=152d:0578:u/' /boot/firmware/cmdline.txt
sudo reboot
```

(`cmdline.txt` is one line; on older images it's `/boot/cmdline.txt`.) Throughput drops slightly but stability returns.

Useful logs

```
dmesg -T | tail -50
journalctl -u smbd -n 100
journalctl -u nginx -u openmediavault-engined -n 100
sudo omv-salt deploy run samba      # re-apply Samba config from OMV
```

Reset OMV admin password (lost)

```
sudo omv-firstaid      # choose "Change control panel administrator password"
```

Scripts Reference

All scripts are Bash, require root, and use `set -euo pipefail`. The disk scripts refuse to run without an explicit device argument and a typed confirmation.

`scripts/encryption-setup.sh` (recommended)

Creates an encrypted data disk.

```
sudo ./scripts/encryption-setup.sh <device> [mapper-name] [label]
sudo ./scripts/encryption-setup.sh /dev/sda            # → sda1-crypt, label nasdata
```

Steps: shows `lsblk` for the device → requires typing the device name to confirm → refuses if any partition is mounted → GPT + one partition → `cryptsetup luksFormat --type luks2` (prompts for passphrase) → `cryptsetup open` → `mkfs.ext4 -L <label>` → prints UUIDs and reminds you to back up the LUKS header.

Afterwards, mount it in OMV (*Storage → File Systems → Mount*).

`scripts/disk-setup.sh`

Creates an **unencrypted** ext4 disk (e.g. a backup drive that never leaves the house). Do not run on a disk you just encrypted.

```
sudo ./scripts/disk-setup.sh <device> [label]
```

Steps: confirm → GPT + one partition → `mkfs.ext4` → prints UUID. It deliberately does not mount or edit `fstab`; let OMV mount it so OMV stays the source of truth.

`scripts/backup-cron.sh`

Nightly rsync of data disk to backup disk.

Variable	Default	Meaning
`SOURCE`	`/mnt/nas-storage/`	Data root (trailing slash = copy contents)
`DEST`	`/mnt/nas-backup`	Backup root
`SNAPSHOTS`	`0`	`1` = dated hard-link snapshots + `current` symlink
`KEEP_DAYS`	`14`	Snapshots to keep when `SNAPSHOTS=1`
`LOG`	`/var/log/nas-backup.log`	Log file

Safety: exits if either path isn't a separate mountpoint (prevents filling the SD card), uses a lock file to prevent overlapping runs, logs start/end/exit code.

Exit codes: `0` success; `1` precondition failed (not mounted, already running); otherwise rsync's code (`23`/`24` = partial transfer / vanished files).

Override per run: `sudo SNAPSHOTS=1 KEEP_DAYS=30 /usr/local/sbin/nas-backup`

Changes from the original scripts

Original	Problem	Fixed
Hard-coded `/dev/sda1`	Device names change; could wipe the wrong disk	Device is a required argument, with confirmation
`mkfs` on a partition that may not exist	Fails on a fresh disk	Script creates the partition table
`disk-setup.sh` + `encryption-setup.sh` both format `sda1`	Running both destroys one or the other	Documented as alternatives
Mount at `/mnt/nas-storage` with `mount`	Not persistent; bypasses OMV	Mount via OMV; optional symlinks
`rsync $SOURCE $DEST` without trailing slash/quotes	Nested directory; word-splitting	Quoted, trailing slash
No mount check in backup	Backs up into the SD card if disk missing	`mountpoint -q` checks
No logging	Silent failures	Log file + exit codes

Script: scripts/backup-cron.sh

```
        #!/usr/bin/env bash
        # Nightly NAS backup: data disk -> backup disk via rsync.
        # Install: sudo install -m 0750 backup-cron.sh /usr/local/sbin/nas-backup
        set -euo pipefail

        SOURCE="${SOURCE:-/mnt/nas-storage/}"
        DEST="${DEST:-/mnt/nas-backup}"
        SNAPSHOTS="${SNAPSHOTS:-0}"
        KEEP_DAYS="${KEEP_DAYS:-14}"
        LOG="${LOG:-/var/log/nas-backup.log}"
        LOCK="/run/nas-backup.lock"

        log(){ echo "$(date '+%F %T') $*" | tee -a "$LOG"; }

        exec 9>"$LOCK"; flock -n 9 || { log "FAILED: another backup is running"; exit 1; }

        # Resolve symlinks and require both to be real, separate mounts
        SRC_REAL="$(readlink -f "${SOURCE%/}")"; DST_REAL="$(readlink -f "$DEST")"
        mountpoint -q "$SRC_REAL" || { log "FAILED: source $SRC_REAL not mounted (LUKS locked?)";
exit 1; }
        mountpoint -q "$DST_REAL" || { log "FAILED: destination $DST_REAL not mounted"; exit 1; }

        OPTS=(-aHAX --delete --numeric-ids --exclude='lost+found' --exclude='.recycle/' --stats)
        log "START $SRC_REAL/ -> $DST_REAL (snapshots=$SNAPSHOTS)"

        set +e
        if [[ "$SNAPSHOTS" == "1" ]]; then
          TODAY="$(date +%F)"; TARGET="$DST_REAL/$TODAY"
          [[ -d "$DST_REAL/current" ]] && OPTS+=(--link-dest="$DST_REAL/current")
          rsync "${OPTS[@]}" "$SRC_REAL/" "$TARGET/" >>"$LOG" 2>&1; RC=$?
          if [[ $RC -eq 0 || $RC -eq 24 ]]; then
            ln -sfn "$TODAY" "$DST_REAL/current"
            find "$DST_REAL" -mindepth 1 -maxdepth 1 -type d -regextype posix-extended \
              -regex '.*/[0-9]{4}-[0-9]{2}-[0-9]{2}' -mtime +"$KEEP_DAYS" -exec rm -rf {} +
          fi
        else
          rsync "${OPTS[@]}" "$SRC_REAL/" "$DST_REAL/current/" >>"$LOG" 2>&1; RC=$?
        fi
        set -e

        if [[ $RC -eq 0 || $RC -eq 24 ]]; then log "OK (rsync exit $RC)"; else log "FAILED (rsync
exit $RC)"; fi
        exit $RC
```

Script: scripts/disk-setup.sh

```
        #!/usr/bin/env bash
        # Create an UNENCRYPTED ext4 disk (e.g. backup drive). DESTROYS ALL DATA ON <device>.
        # Do not run on a disk prepared with encryption-setup.sh.
        # Usage: sudo ./disk-setup.sh /dev/sdX [label]
        set -euo pipefail

        DEV="${1:-}"; [[ -b "$DEV" ]] || { echo "Usage: $0 /dev/sdX [label]"; exit 1; }
        [[ $EUID -eq 0 ]] || { echo "Run as root"; exit 1; }
        LABEL="${2:-nasbackup}"

        lsblk -o NAME,SIZE,MODEL,SERIAL,FSTYPE,MOUNTPOINT "$DEV"
        if lsblk -no MOUNTPOINT "$DEV" | grep -q .; then echo "Device has mounted partitions.
Aborting."; exit 1; fi
        read -rp "Type '$DEV' to ERASE it: " ok; [[ "$ok" == "$DEV" ]] || { echo "Aborted"; exit
1; }

        wipefs -a "$DEV"
        parted -s "$DEV" mklabel gpt mkpart primary ext4 1MiB 100%
        partprobe "$DEV"; udevadm settle
        PART="$(lsblk -lnpo NAME "$DEV" | sed -n 2p)"
        mkfs.ext4 -L "$LABEL" "$PART"

        echo "Done. UUID: $(blkid -s UUID -o value "$PART")"
        echo "Next: mount it in OMV (Storage > File Systems)."
```

Script: scripts/encryption-setup.sh

```
        #!/usr/bin/env bash
        # Create a LUKS2-encrypted ext4 data disk. DESTROYS ALL DATA ON <device>.
        # Usage: sudo ./encryption-setup.sh /dev/sdX [mapper-name] [label]
        set -euo pipefail

        DEV="${1:-}"; [[ -b "$DEV" ]] || { echo "Usage: $0 /dev/sdX [mapper] [label]"; exit 1; }
        [[ $EUID -eq 0 ]] || { echo "Run as root"; exit 1; }
        BASE="$(basename "$DEV")"
        MAPPER="${2:-${BASE}1-crypt}"
        LABEL="${3:-nasdata}"

        lsblk -o NAME,SIZE,MODEL,SERIAL,FSTYPE,MOUNTPOINT "$DEV"
        if lsblk -no MOUNTPOINT "$DEV" | grep -q .; then echo "Device has mounted partitions.
Aborting."; exit 1; fi
        read -rp "Type '$DEV' to ERASE and encrypt it: " ok; [[ "$ok" == "$DEV" ]] || { echo
"Aborted"; exit 1; }

        wipefs -a "$DEV"
        parted -s "$DEV" mklabel gpt mkpart primary 1MiB 100%
        partprobe "$DEV"; udevadm settle
        PART="$(lsblk -lnpo NAME "$DEV" | sed -n 2p)"

        echo "Formatting LUKS2 on $PART (you will be asked for a passphrase)..."
        cryptsetup luksFormat --type luks2 "$PART"
        cryptsetup open "$PART" "$MAPPER"
        mkfs.ext4 -L "$LABEL" "/dev/mapper/$MAPPER"

        echo
        echo "Done."
        echo "LUKS UUID: $(blkid -s UUID -o value "$PART")"
        echo "ext4 UUID: $(blkid -s UUID -o value "/dev/mapper/$MAPPER")"
        echo "Next: mount it in OMV (Storage > File Systems)."
        echo "IMPORTANT: back up the header: cryptsetup luksHeaderBackup
$PART --header-backup-file <file>"
```